

ChatGPT



The screenshot shows a video call with a man on the left and a code editor on the right. The code editor displays C++ code for Selection Sort. The code includes a function `selection_sort` that takes an array and its size, and a `main` function that reads input, calls `selection_sort`, and prints the output. The input is `13 46 24 52 20 9` and the output is `9 13 20 24 46 52`.

```

1 #include<iostream>
2 using namespace std;
3 void selection_sort(int arr[], int n) {
4     for(int i = 0; i < n-1; i++) {
5         int min = i;
6         for(int j = i+1; j < n; j++) {
7             if(arr[j] < arr[min]) {
8                 min = j;
9             }
10        }
11        int temp = arr[min];
12        arr[min] = arr[i];
13        arr[i] = temp;
14    }
15 }
16 int main() {
17     int n;
18     cin >> n;
19     int arr[n];
20     for(int i = 0; i < n; i++) cin >> arr[i];
21     selection_sort(arr, n);
22     for(int i = 0; i < n; i++) {
23         cout << arr[i] << " ";
24     }
25     return 0;
26 }

```

Teach me Selection Sort and give code in java

Compare with other AI ⚡

Absolutely. Let's learn **Selection Sort** from the intuition first, then the Java code, and finally walk through the exact example from your screenshot.

1. What is Selection Sort?

Suppose we have:

[13, 46, 24, 52, 20, 9]

The idea is very simple:

Find the smallest element and put it at the beginning. Then find the next smallest element and put it in the next position. Keep repeating.

Think of it like arranging students by height:

1. Find the shortest student → put them first.
2. From the remaining students, find the shortest → put them second.
3. Continue until everyone is sorted.

2. Let's sort your example

Starting array:

[13, 46, 24, 52, 20, 9]

Pass 1

Find the smallest element in the entire array:

9

Swap 9 with the first element 13:

[9, 46, 24, 52, 20, 13]

Now 9 is permanently sorted.

Pass 2

Ignore the 9.

Look at:

[46, 24, 52, 20, 13]

Smallest = 13

Swap 13 with 46:

[9, 13, 24, 52, 20, 46]

Pass 3

Ignore 9, 13.

Look at:

[24, 52, 20, 46]

Smallest = 20

Swap:

[9, 13, 20, 52, 24, 46]

Pass 4

Look at:

[52, 24, 46]

Smallest = 24

Swap:

[9, 13, 20, 24, 52, 46]

Pass 5

Look at:

[52, 46]

Smallest = 46

Swap:

[9, 13, 20, 24, 46, 52]

Done! 

3. The key idea in code

The most important variable is:

```
Java
int minIndex = i;
```

It means:

"For this round, I'm assuming the element at **i** is the smallest."

Then we search the remaining array:

```
Java
for (int j = i + 1; j < n; j++) {
    if (arr[j] < arr[minIndex]) {
        minIndex = j;
    }
}
```

If we find something smaller, we update **minIndex**.

Finally, swap:

```
Java
int temp = arr[i];
arr[i] = arr[minIndex];
arr[minIndex] = temp;
```

That's essentially the entire Selection Sort algorithm.

4. Java implementation

```
Java
import java.util.*;

public class Main {

    static void selectionSort(int[] arr) {

        int n = arr.length;

        // Move through each position
        for (int i = 0; i < n - 1; i++) {

            // Assume current position has the minimum
            int minIndex = i;

            // Find the smallest element in the remaining array
            for (int j = i + 1; j < n; j++) {

                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            // Swap
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}
```

```

    }
}

public static void main(String[] args) {
    int[] arr = {13, 46, 24, 52, 20, 9};
    selectionSort(arr);
    // Print sorted array
    for (int x : arr) {
        System.out.print(x + " ");
    }
}
}

```

Output:

```
9 13 20 24 46 52
```

5. Understand the two loops

This is the part you should remember for DSA.

```

Java
for (int i = 0; i < n - 1; i++)

```

Outer loop = position where we want to place the minimum.

For example:

```

i = 0 → find smallest for position 0
i = 1 → find smallest for position 1
i = 2 → find smallest for position 2
...

```

And:

```

Java
for (int j = i + 1; j < n; j++)

```

Inner loop = search for the minimum.

So mentally remember:

i = where to put the minimum
j = where to search for the minimum

6. Why $j = i + 1$?

Suppose:

```

[9, 46, 24, 52, 20, 13]
↑
i = 0

```

We've already decided that position 0 needs to contain the minimum.

So we don't need to compare `arr[0]` with itself.

We start searching from:

```
j = i + 1
```

Therefore:

```
i = 0 → j starts at 1
i = 1 → j starts at 2
i = 2 → j starts at 3
```

7. Time and space complexity

Selection Sort has:

Time Complexity:

```
O(n²)
```

Why?

Because we repeatedly search through the remaining elements.

Space Complexity:

```
O(1)
```

Because we sort the array **in-place** and only use a few extra variables.

Important DSA facts

Property	Selection Sort
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space	$O(1)$
In-place	Yes
Stable	No, generally



The one-line memory trick

When you see **Selection Sort**, immediately think:

"Select the minimum → swap it with the current position."

And the pseudocode becomes:

```
for i = 0 to n-2
  minIndex = i

  for j = i+1 to n-1
    if arr[j] < arr[minIndex]
      minIndex = j

  swap(arr[i], arr[minIndex])
```

Once this pattern is clear, Selection Sort becomes one of the easiest sorting algorithms to implement. ChatGPT can make mistakes. Check important info.